

Using AI in this course

Using AI in this course

Uso de inteligencia artificial en este curso

The short version

You may use AI to **understand**. You may not use AI to **produce**.

Concretely: ask an assistant to explain what a while loop does, why your error message says what it says, or what the difference is between return and print — that is study, and it is allowed. Ask it to write the exercise, the homework, the lab or any part of your project — that is not your work, and it is an academic integrity violation under the Reglamento Académico, handled by the Campus Integrity Committee, not by me.

Why I care, stated plainly

I am not worried about you cheating. I am worried about you arriving in your fourth semester unable to write twenty lines of code without help, because everything before it was produced rather than learned. Programming is one of the few skills where the shortcut and the destination point in opposite directions: the assistant gets you the working program and costs you the ability to write the next one.

You will use these tools professionally. I use them. The reason you may not use them here is the same reason a pilot logs hours in a simulator before the autopilot: you cannot supervise a tool that is better than you at something you never learned to do.

What is allowed

☐ Allowed	☐ Not allowed
Asking for a concept to be explained differently	Asking for code that solves a set exercise
Pasting an error message and asking what it means	Pasting the problem statement and asking for a solution
Asking why <i>your own</i> code behaves as it does	Asking for your code to be “improved” or “finished”
Generating extra practice problems for yourself	Generating your homework, lab, or project deliverable
Translating a term you do not know into Spanish	Translating a whole solution from somewhere else

If you are unsure which side of the line something sits on, the test is: **could you reproduce it on a blank page, right now, with no device?** If not, it is not yours yet.

The four mechanisms

I would rather build a course where the shortcut does not help than police one where it does. Four things, all of which start in Week 1:

1. The algorithm comes first, on paper

Every exercise in this course — class, lab and homework — begins with the algorithm written by hand in A_Matricula.docx or on paper, **before** any editor is open. I check the handwritten algorithm. The code is graded against *your* algorithm, not against a correct answer.

If your algorithm is naive and your code is elegant, we will talk.

2. Graded work happens in the room

Class activities and lab exercises are submitted during the session. This is academia policy for all TC1037 groups: if you are not in the session, that activity is a zero — there is no lower-weighted late version. Your lab is a supervised start on that week's homework, with the TA present and AI closed; Sunday's deadline is when you submit the finished version, not when you begin it.

AI is not permitted in any class or lab session. That is an academia-wide rule.

3. You will explain your own code, out loud

On each situación problema deliverable and on selected labs, I will ask you — individually, for about two minutes, with the code on screen and nothing in your hands — to walk through a section of your own submission and answer one question about it. Something like: *what happens if this column is empty?* or *why is this a for and not a while?*

This is not a trap and it is not an interrogation. If you wrote it, it takes two minutes and you will find it easy. If you cannot explain your own submission, the submission does not count, regardless of whether it runs.

The final project deliverable is individual precisely so that this works.

4. Exams are closed-environment

Quick exams, the midterm and the final are taken under Respondus LockDown Browser or on paper. Thonny is not available. No external tool, no phone except for the initial authentication, no Macs (an academia restriction, not mine).

This is the honest part: exams are 62% of your grade. If the previous three mechanisms fail and you produce your way through the semester, the exams will find it in November and it will be too late to fix. The mechanisms exist so that does not happen to you.

The one exception, and why it exists

In **Week 10** there is a single in-class activity where using an AI assistant is not just allowed but **required**: you will read a data file and plot it with an assistant's help, and then your actual task is to *critique what it produced* — against a file you already understand, using everything you learned in the nine weeks when you could not use one.

It is there deliberately, and it is institutional: this learning unit is designed with one AI-supported experience built in. It does not weaken the rest of this policy; it is what the rest of this policy is *for*. You cannot evaluate a tool that is better than you at something you never learned to do — so we spend nine weeks learning it, and one session evaluating the tool. That is the order that makes you an engineer rather than an operator.

Everywhere else in this course, the rule above stands.

If you are already behind

The single worst outcome of this policy is a student who falls behind in Week 3, starts generating their homework to keep up, and hides it until the midterm. If that is where you are, come to asesorías. Being behind is ordinary and fixable; arriving at the final exam having never written your own code is neither.

Glosario / Spanish gloss

English	Español
algorithm	algoritmo
statement	sentencia, instrucción
assignment	asignación
variable	variable
built-in function	función predefinida
loop	ciclo
to debug	depurar
output	salida
deliverable	entregable